Monitoring, Support and Maintenance Plan

Project: Predicting 30-day hospital readmission risk in diabetic patients (FUSE AI-201, Group 2)

Status of this document: a *proposed workflow* for running the prototype beyond the demonstration stage. It is a plan, not a running service. Section 6 states exactly which parts exist in this repository today.

Intended use if adopted: a decision-support aid that helps discharge-planning staff decide which diabetic patients to follow up with first. It must never be used to deny or delay care, and every flag is reviewed by a person.

---

1. Performance monitoring

The main difficulty. The true outcome (was the patient readmitted within 30 days?) is only known 30 days after discharge. So we monitor in two layers.

Layer A: immediate checks (every request, reviewed weekly). These need no outcome.

What we watch	Baseline (from our held-out test set)	Warning level	What we do
Share of patients flagged high-risk (score ≥ 0.51)	29.7% (4,414 of 14,859)	more than 5 points away from baseline for 2 weeks	Check for a change in patient mix or a data problem before blaming the model
Tier mix (Low / Medium / High)	about 50% / 35% / 15% (the validation percentiles used to set the tiers)	Population Stability Index (PSI) on the score above 0.2	Same as above; compare inputs
Input drift on the 8 numeric inputs (e.g. prior inpatient visits, length of stay)	training distributions	PSI above 0.2 on any one, or above 0.1 on three or more	Review with the data steward
Missing-value rates	`payer_code` about 40% missing, `medical_specialty` about 49%	change of more than 10 points	Likely an upstream data-feed change; fix at the source
Share of requests with a warning (out-of-range value or unseen category)	0.13% (2 of 1,500 randomly sampled real test rows)	above 2%	Investigate new codes (e.g. a new discharge code)
Latency (95th percentile)	median 9 ms and 95th percentile 14 ms on 1,500 test rows (in-process); 10 to 50 ms through the API. The first request after start-up took 2.7 s because the model loads then	above 2 s (our Module 4 criterion), apart from the first request	Check the host and the logs. Load the model once at start-up so users never hit the slow first call

Layer B: outcome-based checks (monthly, once 30-day outcomes are known).

What we watch	Baseline (test set)	Amber (investigate)	Red (stop and review)
ROC-AUC (rolling 3 months)	0.6856	below 0.66	below 0.62
Recall at the flag threshold	0.529	below 0.45	below 0.40
Precision at the flag threshold	0.201	below 0.17	below 0.14
Share of readmissions captured in the top 20% of scores	0.401	below 0.36	below 0.32
Calibration (score vs. observed readmission rate, by bin)	known to be poor: scores run higher than real rates	the *shape* of the curve changes noticeably	no longer ranks in order
Subgroup recall and false-alarm rate (age band, gender, race, admission type)	single split, noisy	gap between groups above 0.10 with at least 200 patients per group	gap above 0.15

These numbers are **starting values**. A month of data holds far fewer patients than our 14,859-row test set, so single months will be noisy. We would use rolling windows with confidence intervals (bootstrap) and revise the levels after the first few months.

How it runs. The prediction service already writes one JSON line per request with the time, score, tier, number of warnings and latency, and never any patient values (see `api.py`). A weekly script reads those lines for Layer A; a monthly script joins the logged predictions to outcomes for Layer B. Results go to a simple dashboard page (Streamlit or Grafana) and a short monthly review with the model owner and a clinical reviewer.

2. Error handling

Already in the prototype: missing or non-numeric fields never crash a prediction (they are filled in by the pipeline and reported as warnings); out-of-range values and unseen categories produce warnings; malformed requests return HTTP 422; identifiers such as `patient_nbr` are ignored; 23 automated tests cover this.

Planned: - **Application errors:** log every exception with a request ID and no patient values; alert the on-call engineer when the error rate passes 1% in an hour. Show users a plain message, never a stack trace. - **Unexpected predictions:** a prediction is "unexpected" if (a) two near-identical inputs get very different scores, (b) a High tier comes with several input warnings, or (c) the score distribution jumps between weeks. Each month a clinical reviewer checks a random sample of 20 flagged cases for common sense. - **Triage:** P1 (wrong or no predictions, possible patient impact): same-day response and switch the service to "scores unavailable". P2 (degraded quality or a drift alert): fix within 1 week. P3 (cosmetic or documentation): next release. - **Rollback:** keep the last two model versions. If a new version misbehaves, switch back and confirm that the reference cases still return their known scores (typical case 0.4857, higher-risk 0.69, lower-risk 0.2377).

3. Data and model updates

1. Evaluate first. Before retraining, score the most recent 3 to 6 months of discharges with the current model and compare against the baseline above. Often a recalibration (Platt or isotonic) is enough and is far cheaper than a rebuild.

2. Retraining triggers (any one): - Layer B stays amber for two consecutive months, or goes red once. - Input drift is above the PSI levels for a key input (prior inpatient visits, discharge disposition). - A clinical or coding

change: a new discharge programme, new medicines, or a new coding system. Note that our data uses ICD-9 diagnosis codes, whereas hospitals now use ICD-10, so any current data must be mapped before it can be used. - A scheduled yearly review.

3. How a new model is accepted. Re-run the same pipeline (the five candidate models, patient-grouped split), and add a time-based split on the newest data. Compare it with the current "champion" on the same recent patients. Promote it only if (a) ROC-AUC improves by at least 0.01 with an interval that does not cross zero, (b) no subgroup gets worse, (c) calibration is checked, and (d) it runs in "shadow mode" for 4 weeks beside the champion without being shown to staff.

4. Versioning. Each model is saved with its metadata file (features, thresholds, test metrics, library versions), logged in MLflow, and tagged in Git. A change log records the data window, the reason for the update and the results.

5. Governance. New data needs approval from the data owner, must stay de-identified, and is kept only as long as the agreement allows.

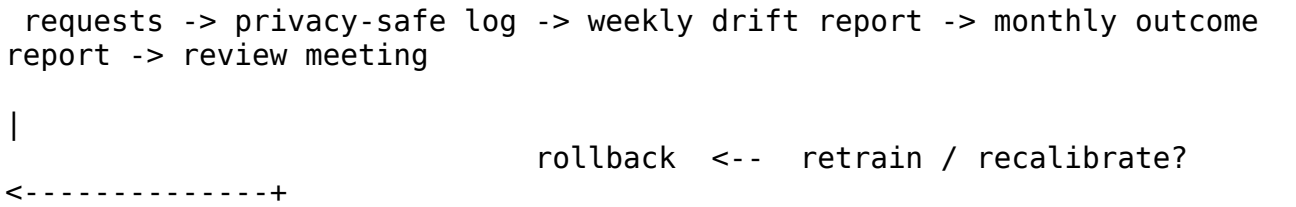
4. Security and responsible AI

Area	Risk	Safeguard
Privacy	The tool handles health information	The prototype uses only public, de-identified data and contains no names or ID numbers; the model never uses identifiers. In real use: send the minimum data needed, encrypt traffic (HTTPS) and stored data, and log no patient values
Access	Anyone could call the service	The demo has no login. A real service needs authentication (hospital single sign-on or API keys), role-based access and an audit trail of who scored what
Secrets	Keys leaked in the repository	No keys, passwords or credentials are in the repository; real ones would live in environment variables or a secret manager
Supply chain	Vulnerable or tampered dependencies	Exact versions are pinned; run `pip-audit` or GitHub Dependabot monthly; the container runs as a non-root user. The saved model is a pickle file, which can run code when loaded, so only load models from our own trusted, version-controlled source
Bias and fairness	Uneven errors across groups; race is a model input	Monitor recall and false-alarm rate by subgroup (Section 1); review whether race should be removed; our current subgroup checks are small and noisy and are not a fairness audit
Harm from wrong scores	A missed high-risk patient, or an unnecessary follow-up	Human review of every flag; the threshold favours catching patients (recall); the score is shown as a relative ranking, not a

Area	Risk	Safeguard
		probability
Misuse	Using the score to limit care	State the allowed use in the interface and documentation; no automatic decisions
Regulation	Clinical software can be regulated	Real use would first need clinical validation on current local data and approval by the hospital's governance and compliance teams. We make no claim of compliance

5. Support and maintenance

- **Recording problems:** GitHub Issues with three templates (bug, data issue, model concern) and labels for severity. Every fix links to its issue.
- **Fixing defects:** work on a branch, open a pull request, run the automated tests (23 today) on every pull request with GitHub Actions, get one review, then tag a release. Urgent fixes use the same path with a faster review.
- **Updating dependencies:** monthly check for security and version updates. Because the model is tied to its scikit-learn and LightGBM versions, any upgrade is followed by a test that the reference cases still return their known scores, and the model is re-saved and re-validated if they do not.
- **Documentation:** README, the plain-language app guide and the report are updated with every release.
- **Roadmap (in priority order):** (1) recalibrate scores; (2) tune XGBoost and LightGBM with cross-validation and compare repeated splits; (3) add features that help first-time high-risk patients (97% of our missed readmissions had 0 or 1 prior inpatient visit); (4) a proper fairness audit; (5) map ICD-10 data and validate on current data; (6) per-patient explanations (SHAP) in the app; (7) authentication, a monitoring dashboard, and deployment through FastAPI and Docker.



6. What exists in this repository today, and what is only planned

Item	Status
Input validation, warnings, error handling	**Implemented and tested** (`src/predict.py`, 23 tests)
Privacy-safe request log (score, tier, warnings, latency)	**Implemented** in `api.py` (no dashboard on top of it)
Experiment tracking	**Implemented**: `scripts/log_experiments_mlflow.py` (MLflow plus CSV log)
Model metadata and versions recorded	**Implemented** (`models/model_metadata.json`)
Drift reports, outcome monitoring, alerts, dashboard	**Planned only** (needs real production data, which this project does not have)
Authentication, HTTPS, audit trail	**Planned only**
GitHub Actions tests, Dependabot, issue templates	**Planned only**

Item	Status
Retraining pipeline	The model-building code exists in the notebook; the automated retraining workflow is **planned only**